

Unity is strength — A story of model composition

 Loïc Le Bel [Follow](#) 5 min read · Oct 8, 2020



Introduction

Earlier this summer (August 2020), Criteo R&D teams proudly delivered the project codename “DRACuLa” into production, bringing a strong uplift of performance to our engine on our main business cases.

While trends are clearly leaning towards the “A.I.” (Artificial Intelligence) hype, this project demonstrates that a composition of simpler techniques can still beat the deep nets magic in specific cases.



Photo by [Stephan Henning](#) on Unsplash

Genesis — Why?

Since 2010, Machine Learning (ML) has been instrumental in Criteo business. While the type of ML model has its importance (Regression, Deep Neural Network, Decision Trees, ...), two other major steps are of uttermost impact on the performance of the overall pipeline: the **quality of the data** collected in the first place and the so-called **feature engineering**. The later encompasses all **data processing** turning the history of interactions with the user, partner, publisher into a real-valued features vector suitable for consumption by the ML models themselves.

On the one hand, *Criteo's initial success relied on the Generalized Linear Model* (a.k.a. **Logistic Regression**) for ML model and **hashing-trick** for feature encoding. While these tools enable Criteo to scale up and out reliably, *their ML performance is now plateauing*, and we believe Criteo has reached the limits of this architecture: **finding useful features to keep adding to the model becomes increasingly difficult**.

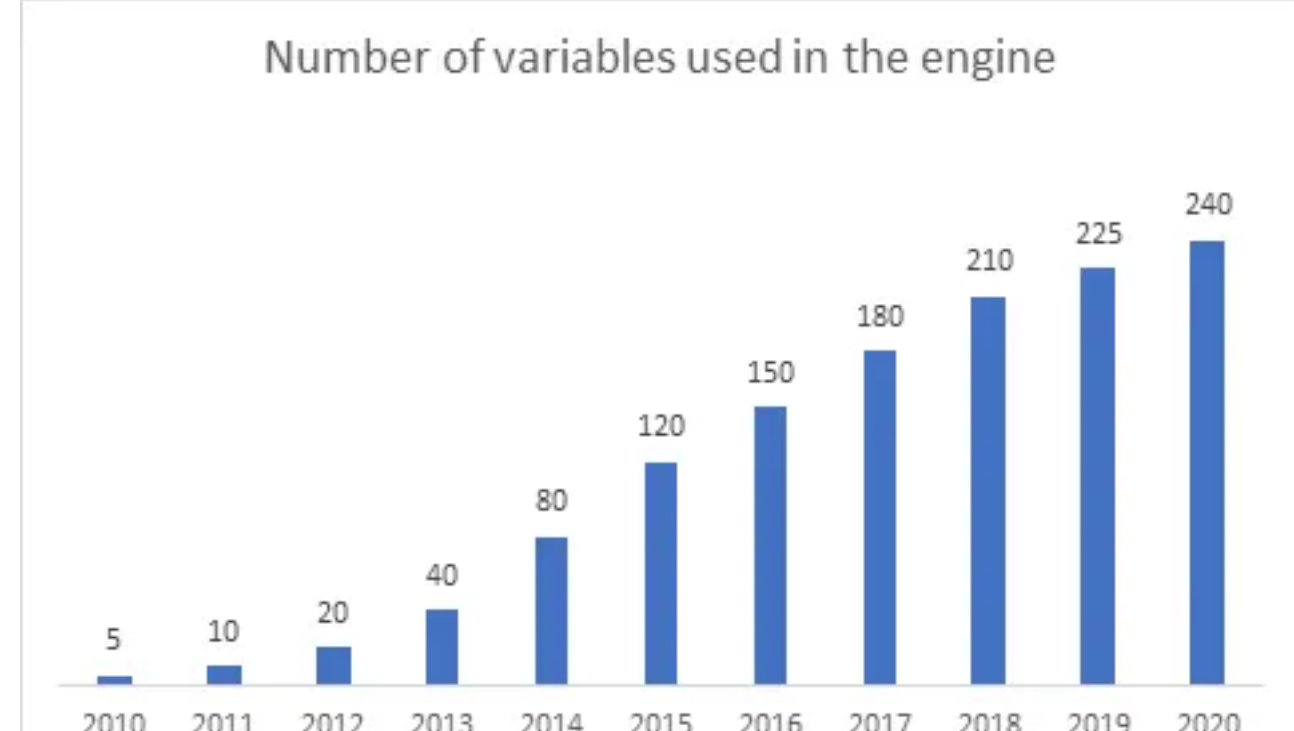


Figure 1: Number of features in model input over time

On the other hand **features formed using counters** of user events on various key dimensions and cross-dimensions **combined with non-linear ML models** are well known to **yield improvements in performance**. This architecture named DRACuLa (Distributed Robust Algorithm for Count Based Learning) has become an *industrial state-of-the-art solution for search models and bidding models* in advertising. It also relies on *approximate counting with count min-sketch (CMS)* to make the counting scalable and the models deployable online.

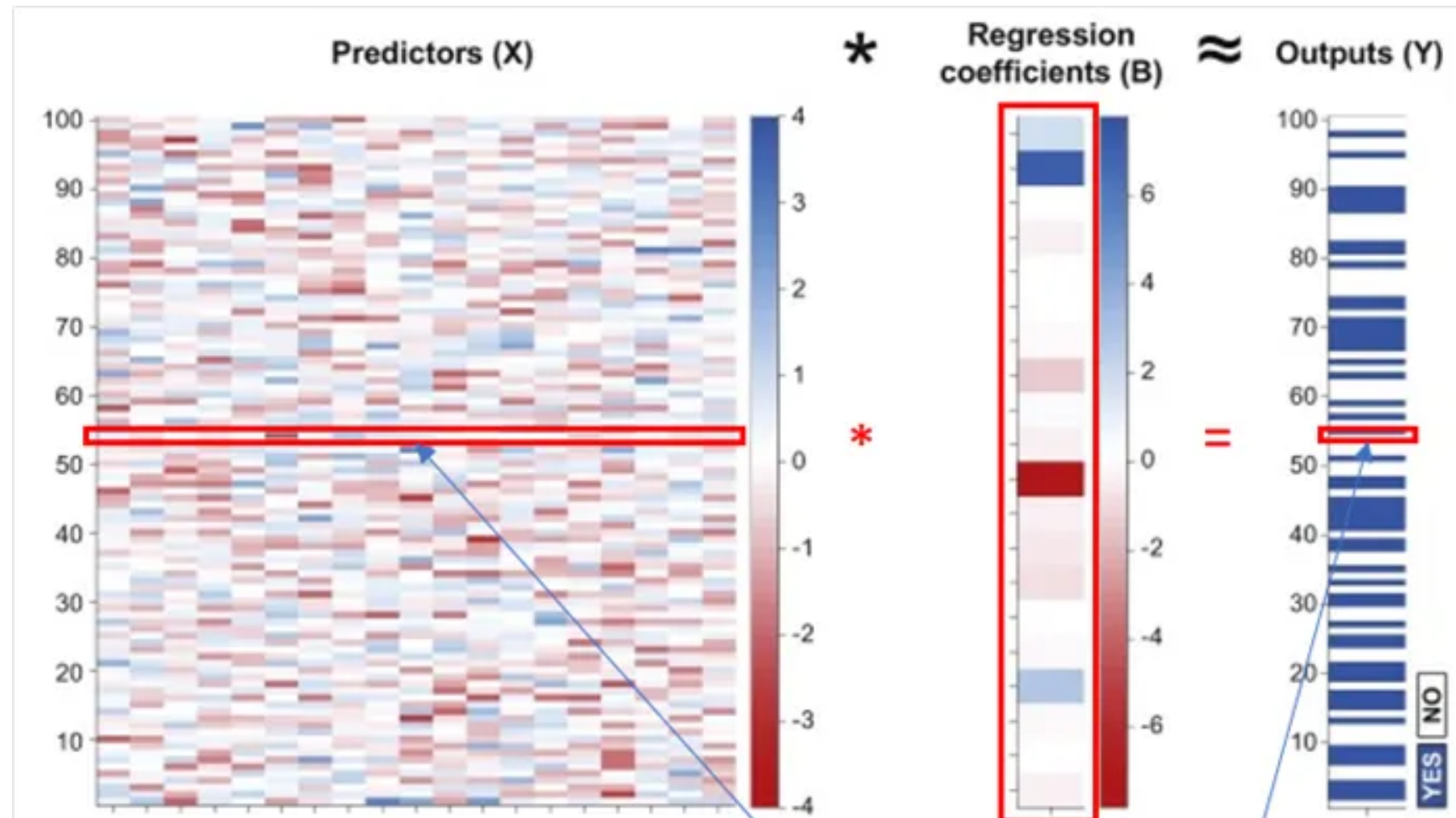
Anatomy — What?

Let us now review the characteristics difference between these two ML model families.

A- Logistic regression model

The historical model used at Criteo are the logistic regression ones: solving the Click Through Rate (CTR) problem by *predicting the probability of a user to click on a displayed banner* given the context. They have the nice properties to be **easily computed** (convex loss), to be able to operate in a high dimensional space using the **hashing trick** (thus **handling high cardinality categorical features** such as IDs — advertiser_id, publisher_id) and to be **very fast at inference time** (only needing simple dot product as seen on figure below). The huge number of examples available at training time (a few billion per day) is counterbalancing the high dimensionality of representation of the features to avoid overfitting at training time.

A simplistic way to think about these models is that they are very efficient at memorizing a given problem when exposed to enough data.



The counterpart of this model family is the *need for manual feature engineering* on the data to achieve the best performance: numerical features must be bucketized to circumvent the linearity of the model, feature interactions (cross- features) must be found and specified explicitly by the data scientists.

B- DRACuLa models (decision trees + counters)

While **decision trees** (DT) are not a novelty (used in expert system for years), their application to Criteo's context and available data is quite challenging: categorical features with many modalities are hard to handle (they would require very high depth of the trees).

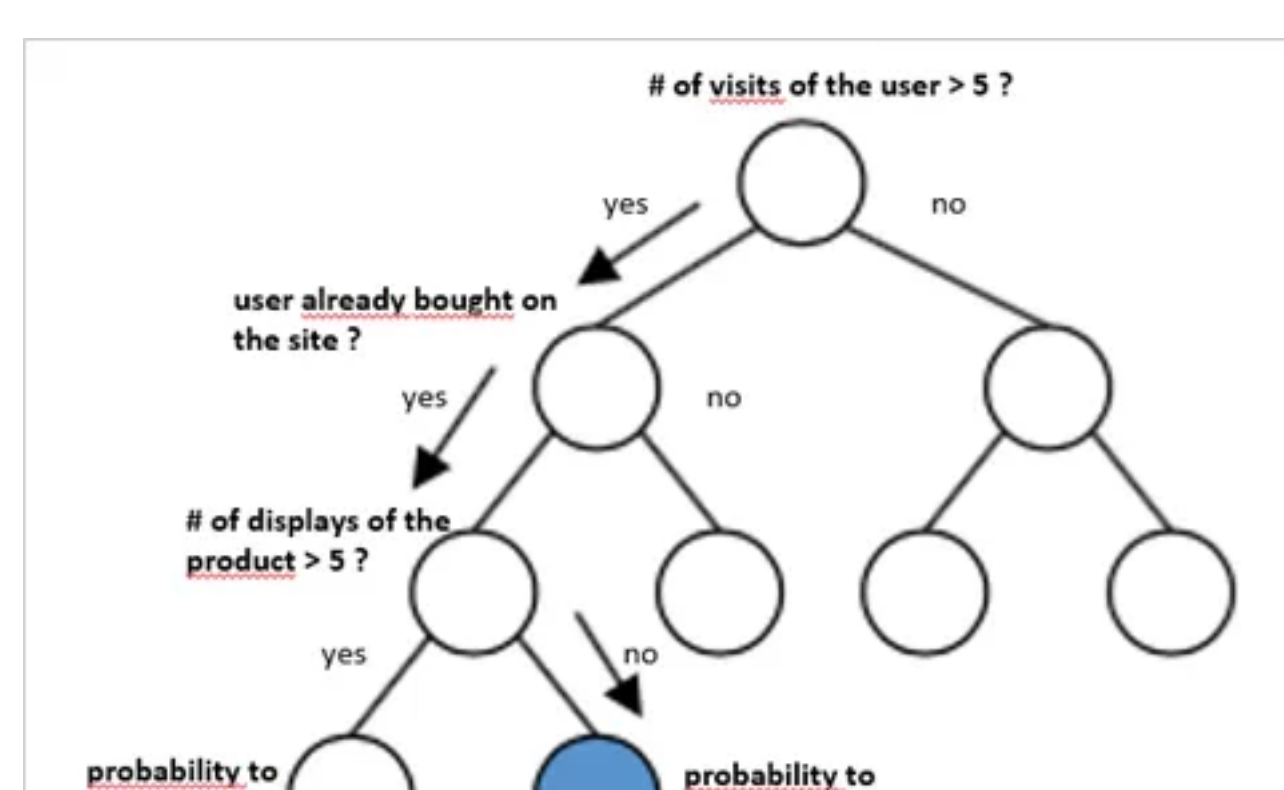


Figure 3: Example of decision tree

The DRACuLa models are solving this issue by combining the decision trees with a specific encoding of the categorical features into numerical ones using mean encoding techniques: instead of using the specific modality of a given input dimension, we use the counts and ratio of the modality in regard of the target we want to optimize (eg.: click or non-click).

partnerid	partnerid_ratio_7d
50784	0.013448
6561	0.043323
22811	0.036398
29715	0.031021
7485	0.007180

Table 1: Mean encoding of partnerid

C- Comparing the characteristics of each model type

In the table below, we summarize the main aspects of both model families:

	Generalized Linear Models	DRACuLa
Feature engineering	- Continuous feature are <i>manually</i> bucketized - Cross-feature must be assessed and added manually - Lots of EDA work just to leverage existing features	- Continuous features are passed to the ML model (handled automatically) - Automatic discovery of cross-feature by the model is possible - EDA can focus on high value tuning and new features
Feature Vector	Huge (2^{25} or 2^6), Sparse, mostly binary (indicator variables)	Small (100s), Dense, quantitative variables
Adaptability	Model training is relatively simple and fast	Model training is relatively complex and slow

Table 2: Model family comparison

Wrap up — How it went?

Our first hope was that the DRACuLa models would just plain outmatch the good old logistic regression ones: ability to express non-linear behavior & autodetection of cross-features were shaping a very promising challenger. First series of online A/B tests (pure DRACuLa vs. Logistic regression) turned out to be negative to neutral. Indeed, beating years of fine tuning and of feature engineering was not going to be an easy fight. Even if disappointing at first for our teams, they did not give up that easily: if the new models could not just outperform the old ones, would they be able to compensate each other's weaknesses?

Following this idea a composition (here geometric mean) of both models was shaped in place using **ensemble methods** leaning on the hypothesis that using multiple learning algorithms would obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone.

History proved this right, beating the production baseline with a whopping uplift on the whole scope of adaptative models!

Unity is strength — Open, Together, Impactful!



Credits to all R&D teams involved: ML Platform, Bidding, CAML, Arbitrage/PRE-Supply/Infra & EPMS

...

...